



中国科学院大学

University of Chinese Academy of Sciences

深度学习 (Deep Learning)

深度学习基础知识

张新峰

计算机科学与技术学院

中国科学院大学

邮箱: xfzhang@ucas.ac.cn



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



提纲

- 数学基础
- 机器学习基础
- 神经元模型
- 感知器及多层感知器
- BackPropagation算法
- 中英文术语对照

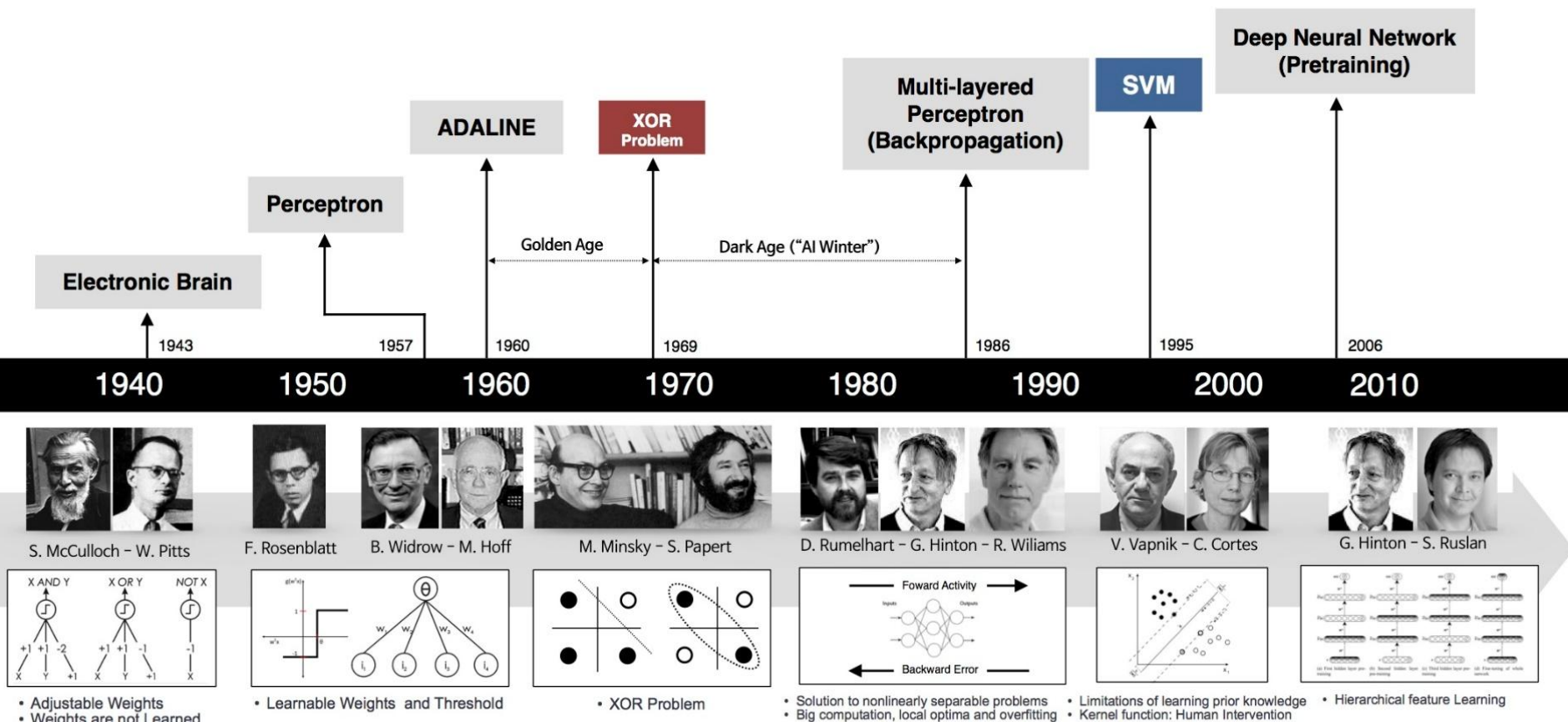


3

神经元模型



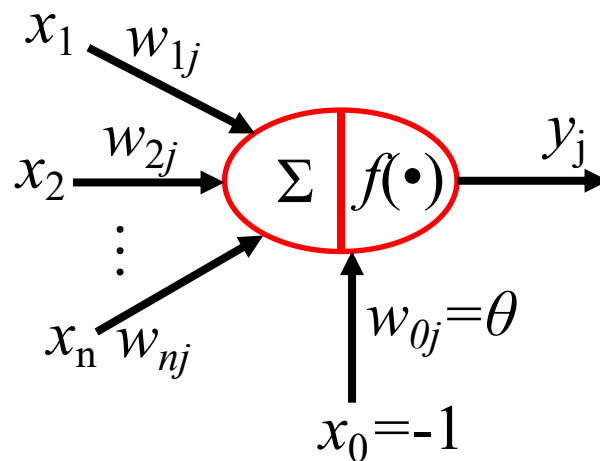
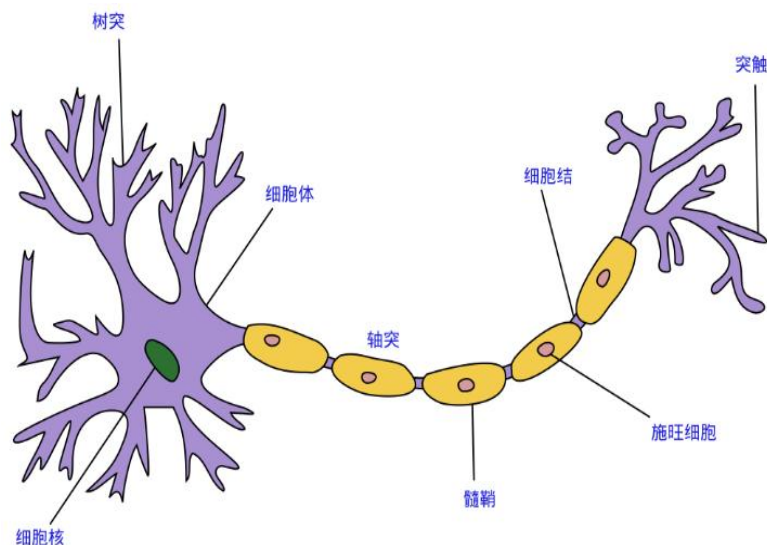
神经网络的发展史



神经元模型

□ M-P模型

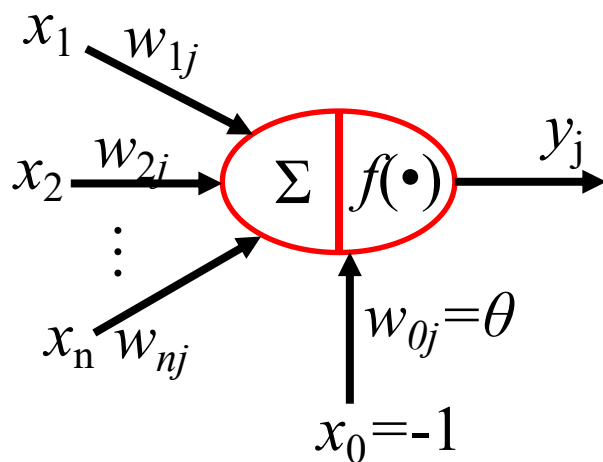
- 1943 年，美国神经生理学家沃伦·**麦卡洛克**（Warren McCulloch）和数学家沃尔特·**皮茨**（Walter Pitts）对生物神经元进行建模，首次提出了一种神经元模型，并命名为McCulloch-Pitts模型，即后来广为人知的**M-P模型**



神经元模型

□ M-P模型

- 在M-P模型中，神经元接受其他n个神经元的输入信号（0或1），这些输入信号经过权重加权并求和，将求和结果与阈值(threshold) θ 比较，然后经过激活函数处理，得到神经元的输出



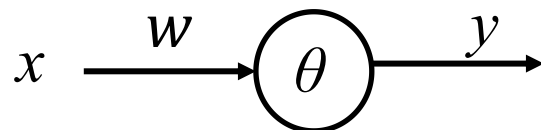
$$y = f\left(\sum_{i=1}^n \omega_{ij} x_i - \theta\right)$$



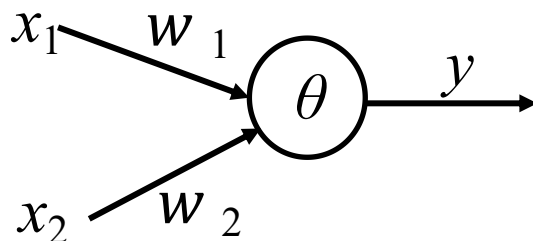
神经元模型

□ M-P 模型可以表示多种逻辑运算

- **取反运算**可以用单输入单输出模型表示，即如果输入为0则输出1，如果输入为1则输出0。由M-P模型的运算规则可得 $w = -2$ ， $\theta = -1$



- **逻辑或**与**逻辑与**运算可以用双输入单输出模型表示。以**逻辑与**运算为例， $w_1 = 1$ ， $w_2 = 1$ ， $\theta = 1.5$



当时还没有通过对训练样本进行训练来确定参数的方法，上述参数只能人为事先计算后确定





4

感知器及多层感知器



感知器

□ 感知器

- 1958 年，罗森布拉特（Roseblatt）提出了感知器，与 M-P 模型需要人为确定参数不同，感知器能够通过训练自动确定参数。训练方式为有监督学习，即需要设定训练样本和期望输出，然后调整实际输出和期望输出之差的方式（误差修正学习）

$$w_i \leftarrow w_i + \alpha(r - y)x_i$$

$$\theta \leftarrow \theta - \alpha(r - y)$$

其中， α 是学习率， r 和 y 分别是期望输出和实际输出。



感知器

□ 感知器权重调整的基本思路

- 实际输出 y 与期望输出 r 相等时, w 和 θ 不变
- 实际输出 y 与期望输出 r 不相等时, 调整 w 和 θ 的值

$$w_i \leftarrow w_i + \alpha(r - y)x_i$$

$$\theta \leftarrow \theta - \alpha(r - y)$$

实际输出 y 和期望输出 r 不相等时的调整策略

实际输出 $y = 0$, 期望输出 $r = 1$ 时
(未激活)

- 减小 θ
- 增大 $x_i = 1$ 的连接权重 w_i ,
- $x_i = 0$ 的连接权重不变

实际输出 $y = 1$, 期望输出 $r = 0$ 时
(激活过度)

- 增大 θ
- 降低 $x_i = 1$ 的连接权重 w_i ,
- $x_i = 0$ 的连接权重不变



感知器

□ 感知器训练过程

0. 训练准备

- 准备 N 个训练样本 x_i 和期望输出 r_i
- 初始化参数 ω_i 和 θ

1. 调整参数

1.1. 迭代调整，直到误差为0或小于某个指定数值

1.1.1. 逐个加入训练样本，计算实际输出

：实际输出和期望输出相等时，参数不变

：实际输出和期望输出不同时，通过误差修正学习调整参数

重复上述步骤 1.1.1

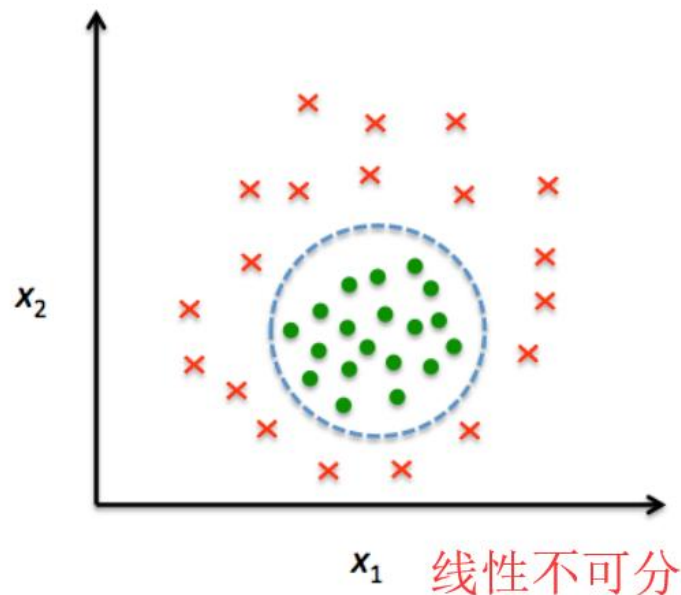
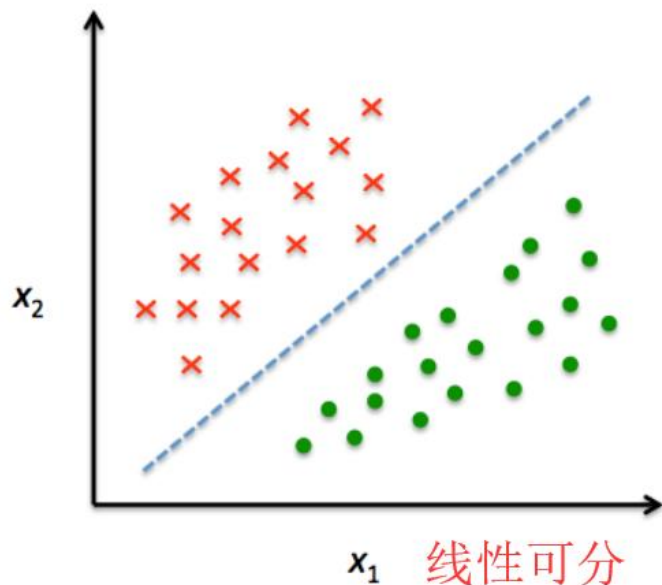
重复上述步骤1.1



多层感知器

□ 单层感知器只能解决线性可分问题，而不能解决线性不可分问题，为了解决线性不可分问题，我们需要使用多层感知器

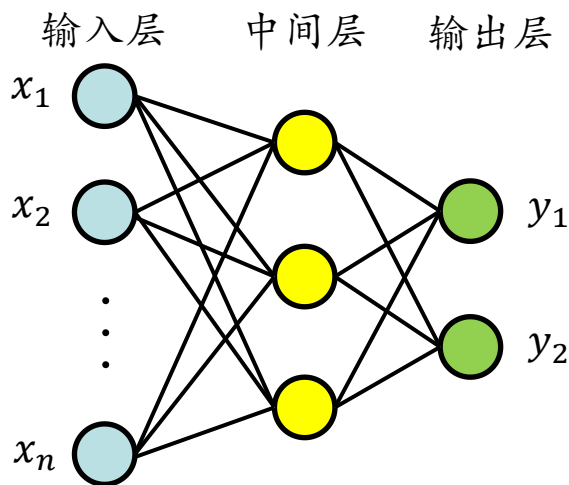
$$y = f\left(\sum_{i=1}^n \omega_{ij}x_i - \theta\right)$$



多层感知器

□ 多层感知器指的是由多层结构的感知器递阶组成的输入值向前传播的网络，也被称为**前馈网络**或**正向传播网络**

- 以三层结构的多层感知器为例，它由**输入层**、**中间层**及**输出层**组成
 - 与M-P 模型相同，中间层的感知器通过权重与输入层的各单元相连接，通过阈值函数计算中间层各单元的输出值
 - 中间层与输出层之间同样是通过权重相连接





5

BackPropagation (BP)算法



BP算法

□ 多层感知器的训练使用**误差反向传播算法**（Error Back Propagation），即**BP算法**

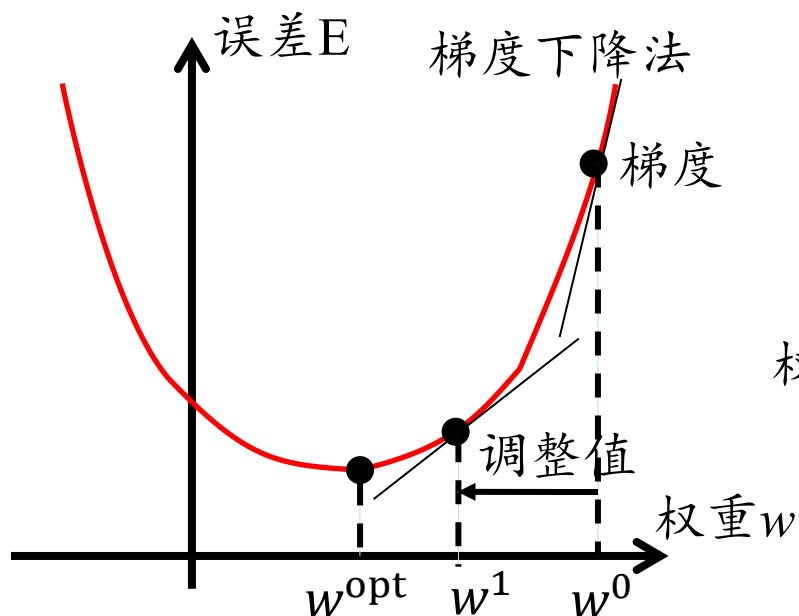
- BP算法最早由沃博斯于1974年提出，鲁梅尔哈特等人进一步发展了该理论
- 基本过程：
 - 前向传播计算：由输入层经过隐含层向输出层的计算网络输出
 - 误差反向逐层传递：网络的期望输出与实际输出之差的误差信号由输出层经过隐含层逐层向输入层传递
 - 由“前向传播计算”与“误差反向逐层传递”的反复进行的网络训练过程



BP算法

□ 多层感知器的训练使用**误差反向传播算法**（Error Back Propagation），即**BP算法**

- BP算法就是通过比较实际输出和期望输出得到误差信号，把误差信号从输出层逐层向前传播得到各层的误差信号，再通过调整各层的连接权重以减小误差。权重的调整主要使用梯度下降法



$$\text{误差公式: } E = \sum_{n=1}^N \|r_n - y_n\|^2$$

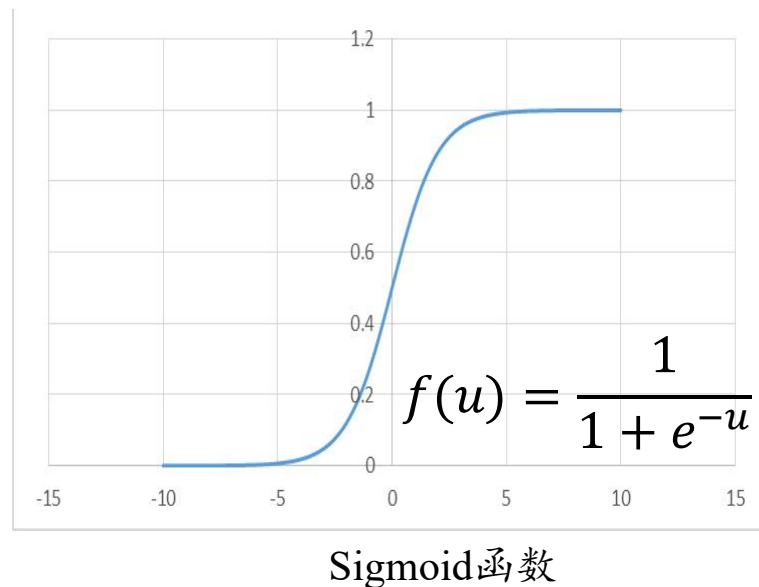
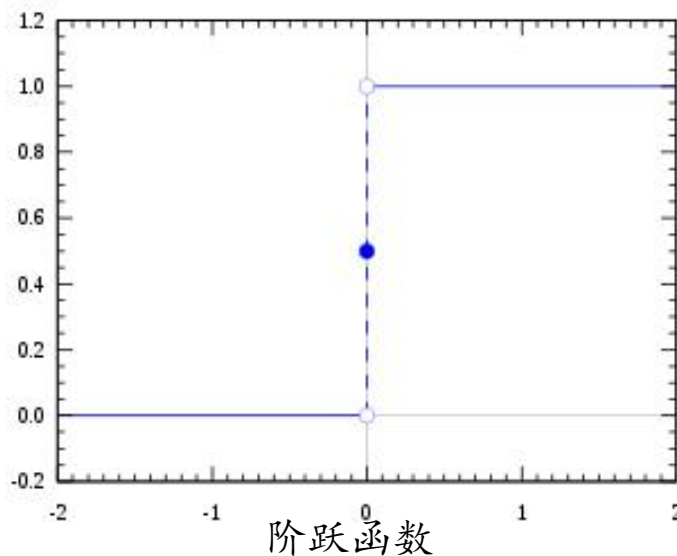
$$\text{权重调整值公式: } \Delta w = -\alpha \frac{\partial E}{\partial w}$$



BP算法

□ BP算法中激活函数

- 通过误差反向传播算法调整多层感知器的连接权重时，一个瓶颈问题就是激活函数
 - M-P 模型中使用阶跃函数作为激活函数，只能输出 0或 1，不连续所以不可导
 - 为了使误差能够传播，鲁梅尔哈特等人提出使用可导函数sigmoid作为激活函数

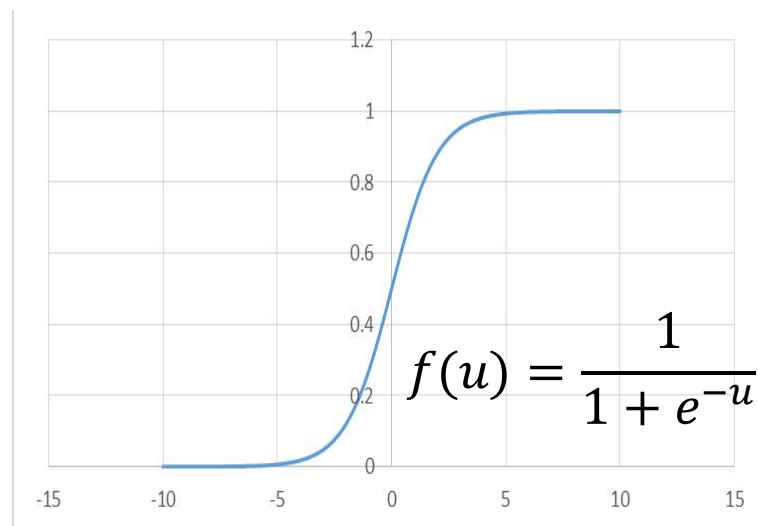


BP算法

□ BP算法中激活函数

– Sigmoid函数的导数

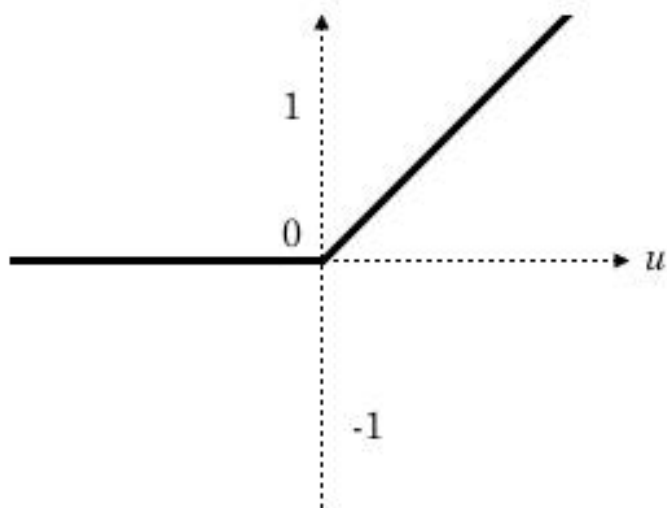
$$\frac{df(u)}{du} = f(u)(1 - f(u))$$



BP算法

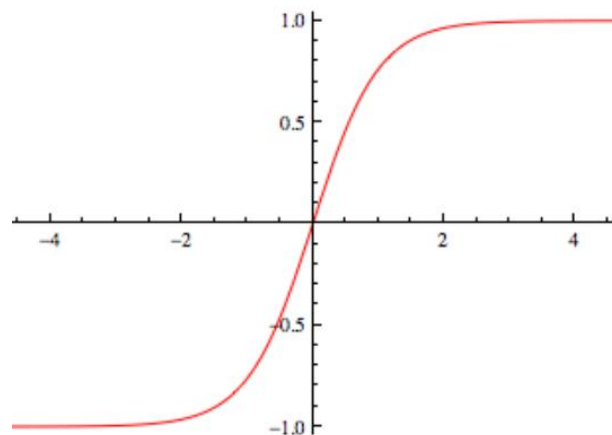
□ 其他常用的激活函数

- ReLU (Rectified Linear Unit, 修正线性单元)和双曲正切tanh等



ReLU

$$\text{ReLU}(x) = \begin{cases} x, & x > 0 \\ 0, & \text{otherwise} \end{cases}$$



tanh

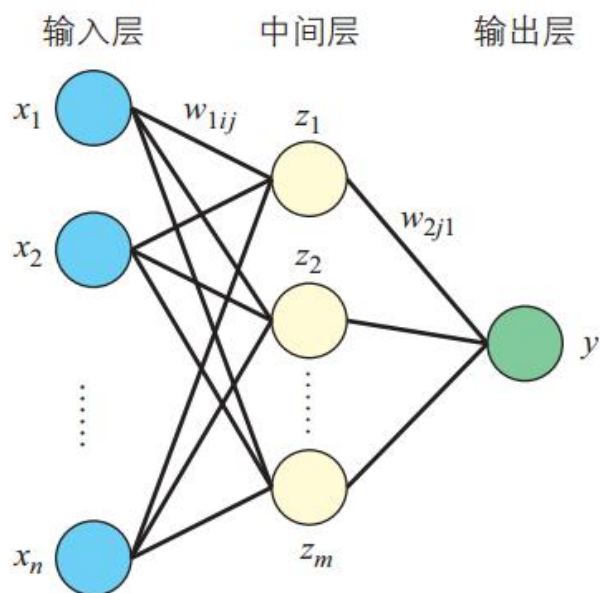
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



BP算法

□ BP算法示例

- 以包含一个中间层和一个输出单元 y 的多层感知器为例： w_{1ij} 表示输入层与中间层之间的连接权重， w_{2j1} 表示中间层与输出层之间的连接权重， i 表示输入层单元， j 表示中间层单元



- 首先调整中间层与输出层之间的连接权重，其中 $y=f(u)$, f 是激活函数， $u_{21} = \sum_{j=1}^m w_{2j1} z_j$ ，把误差函数 E 对连接权重 w_{2j1} 的求导展开成复合函数求导：

$$\begin{aligned}\frac{\partial E}{\partial w_{2j1}} &= \frac{\partial E}{\partial y} \frac{\partial y}{\partial u_{21}} \frac{\partial u_{21}}{\partial w_{2j1}} \\ &= -(r - y)y(1 - y)z_j\end{aligned}$$

这里， z_j 表示的是中间层的值



BP算法

□ 第二，中间层到输出层的连接权重调整值如下所示：

$$\Delta w_{2j1} = \alpha(r - y)y(1 - y)z_j$$

$$w_{2j1} = w_{2j1} + \Delta w_{2j1}$$



BP算法

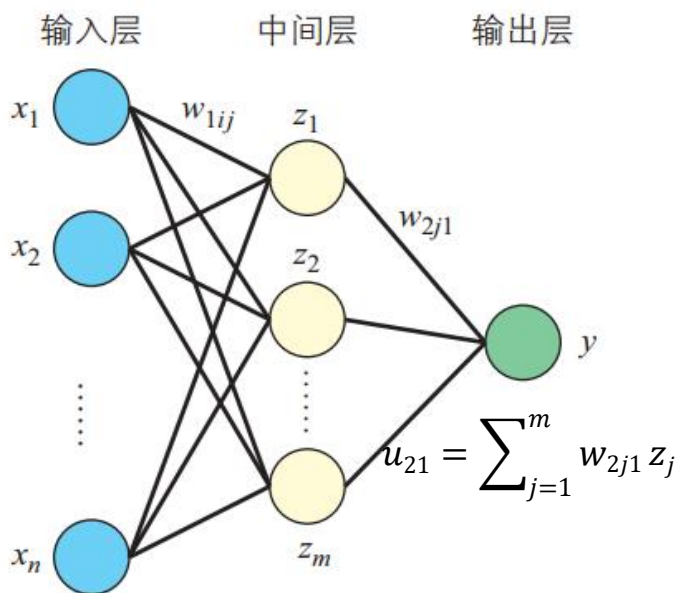
□ 第三，调整输入层与中间层之间的连接权重

- 输入层与中间层之间的连接权重调整值是根据输出层的误差函数确定的，求导公式如下所示：

$$\begin{aligned}\frac{\partial E}{\partial w_{1ij}} &= \frac{\partial E}{\partial y} \frac{\partial y}{\partial u_{21}} \frac{\partial u_{21}}{\partial w_{1ij}} \\ &= -(r - y)y(1 - y) \frac{\partial u_{21}}{\partial w_{1ij}}\end{aligned}$$

其中 u_{21} 由中间层的值 z_j 和连接权重 w_{2j1} 计算得到，中间层与输出层单元之间的激活值 u_{21} 对中间层的值 z_j 求导，结果只和连接权重 w_{2j1} 相关

$$\frac{\partial u_{21}}{\partial w_{1ij}} = \frac{\partial u_{21}}{\partial z_j} \frac{\partial z_j}{\partial w_{1ij}} \quad \frac{\partial u_{21}}{\partial z_j} = w_{2j1}$$



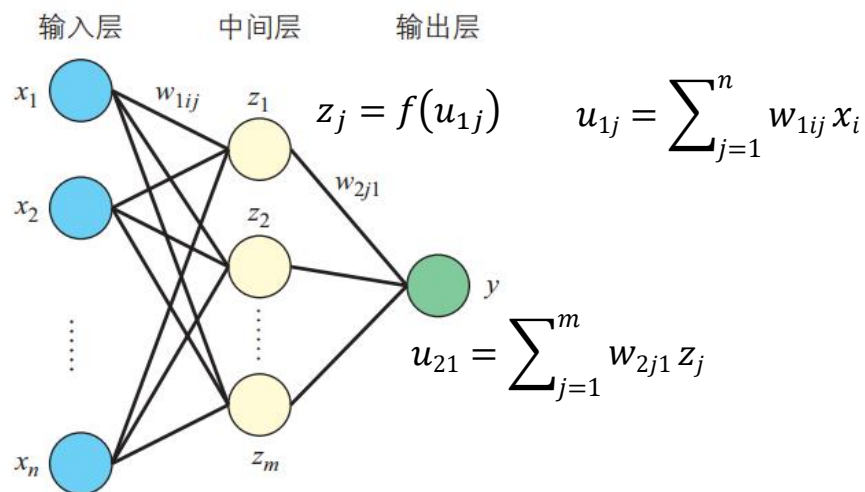
BP算法

□ 中间层的值 z_j 对连接权重 w_{1ij} 求导

$$\frac{\partial z_j}{\partial w_{1ij}} = \frac{\partial z_j}{\partial u_{1j}} \frac{\partial u_{1j}}{\partial w_{1ij}}$$

和 y 一样， z_j 也是sigmoid函数，对 z_j 求导，得到下式：

$$\frac{\partial z_j}{\partial u_{1j}} = z_j(1 - z_j)$$



BP算法

□ 中间层的值 z_j 对连接权重 w_{1ij} 求导

$$\frac{\partial z_j}{\partial w_{1ij}} = \frac{\partial z_j}{\partial u_{1j}} \frac{\partial u_{1j}}{\partial w_{1ij}}$$

和 y 一样， z_j 也是sigmoid函数，对 z_j 求导，得到下式：

$$\frac{\partial z_j}{\partial u_{1j}} = z_j(1 - z_j)$$

输入层与中间层单元之间的激活值 u_{1j} 对中间层与输出层之间的连接权重 w_{1ij} 求导，结果只和 x_i 相关：

$$\frac{\partial u_{1j}}{\partial w_{1ij}} = x_i$$

综上，输入层与中间层之间的链接权重调整值如下：

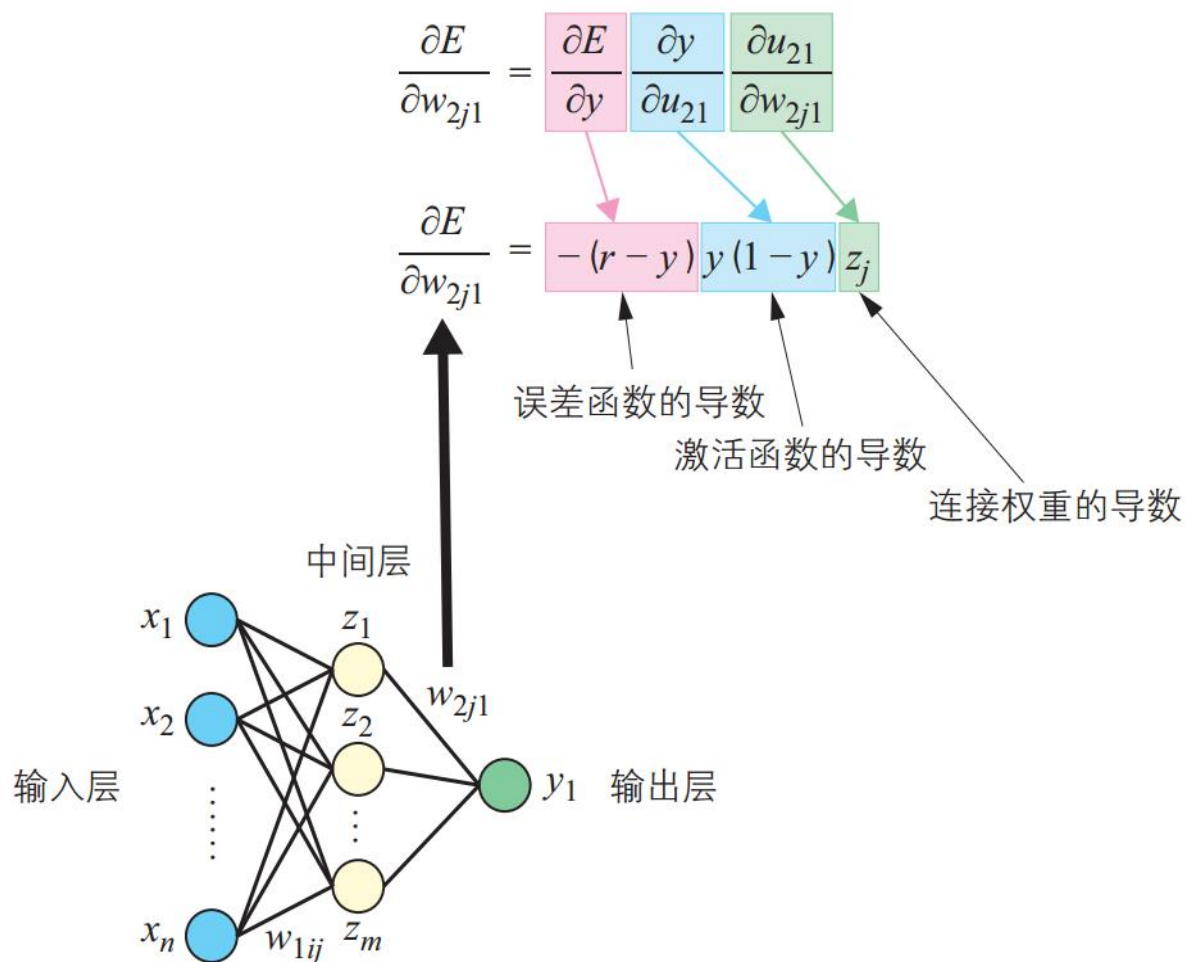
$$\Delta w_{1ij} = \alpha(r - y)y(1 - y)w_{2j1}z_j(1 - z_j)x_i$$

$$w_{1ij} = w_{1ij} + \Delta w_{1ij}$$



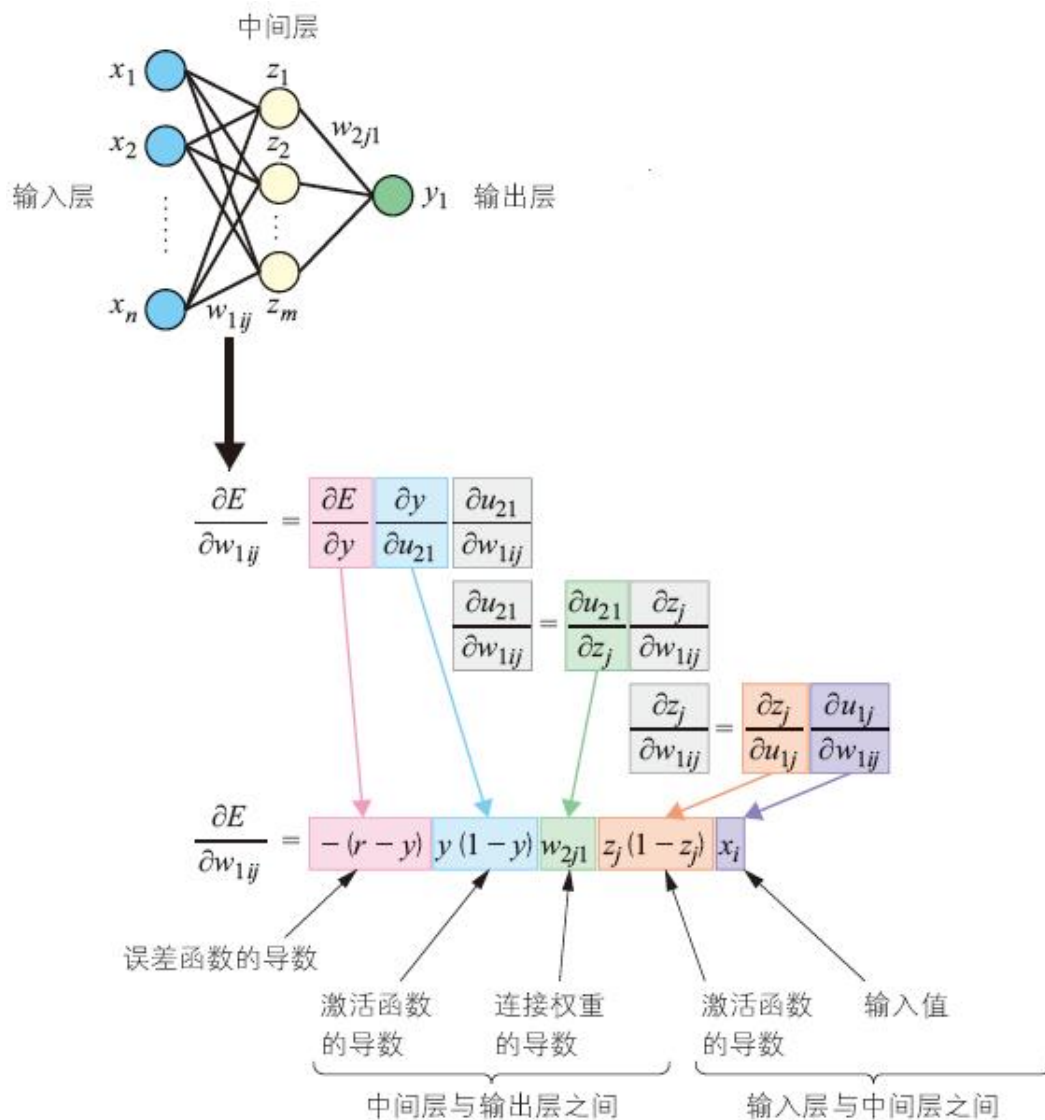
BP算法

□ 中间层到输出层



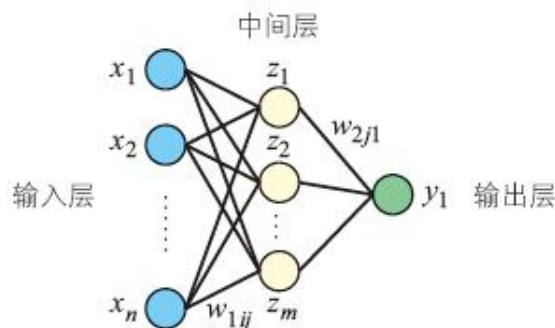
BP算法

□ 输入层到中间层

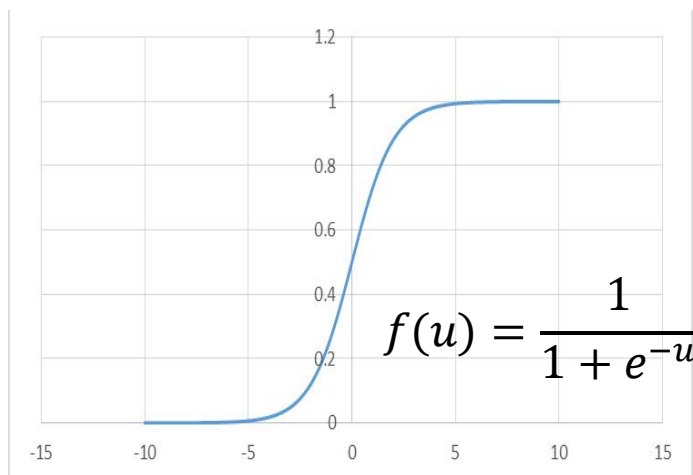


BP算法

□ 输入层到中间层



$$\frac{df(u)}{du} = f(u)(1-f(u))$$



$$\frac{\partial E}{\partial w_{1ij}} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial u_{21}} \frac{\partial u_{21}}{\partial w_{1ij}}$$

$$\frac{\partial u_{21}}{\partial w_{1ij}} = \frac{\partial u_{21}}{\partial z_j} \frac{\partial z_j}{\partial w_{1ij}}$$

$$\frac{\partial z_j}{\partial w_{1ij}} = \frac{\partial z_j}{\partial u_{1j}} \frac{\partial u_{1j}}{\partial w_{1ij}}$$

$$\frac{\partial E}{\partial w_{1ij}} = \underbrace{-(r-y)}_{\text{red}} \underbrace{y(1-y)}_{\text{blue}}$$

误差函数的导数

激活函数的导数

连接权重
的导数

激活函数
的导数

输入值

中间层与输出层之间

输入层与中间层之间





6

中英文术语对照



中英文术语对照

- 标量: **Scalar**
- 向量: **Vector**
- 张量: **Tensor**
- 期望: **Expectation**
- 方差: **Variance**
- 熵: **Entropy**
- 过拟合: **Overfitting**
- 欠拟合: **Underfitting**
- 监督学习: **Supervised learning**
- 无监督学习: **Unsupervised learning**



中英文术语对照

- ❑ 数据集：Data set
- ❑ 训练集：Training set
- ❑ 验证集：Validation set
- ❑ 测试集：Testing set
- ❑ 泛化：Generalization
- ❑ 线性回归：Linear Regression
- ❑ 支持向量机：Support Vector Machine
- ❑ 决策树：Decision Tree
- ❑ 随机森林：Random Forest
- ❑ 感知器：Perceptron



中英文术语对照

- ❑ 反向传播算法: (error) Back Propagation, BP
- ❑ 梯度下降: Gradient Descent
- ❑ 修正线性单元: Rectified Linear Unit, ReLU



谢谢！



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY